

HIGH PERFORMANCE COMPUTING

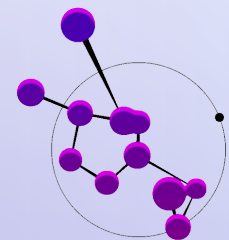
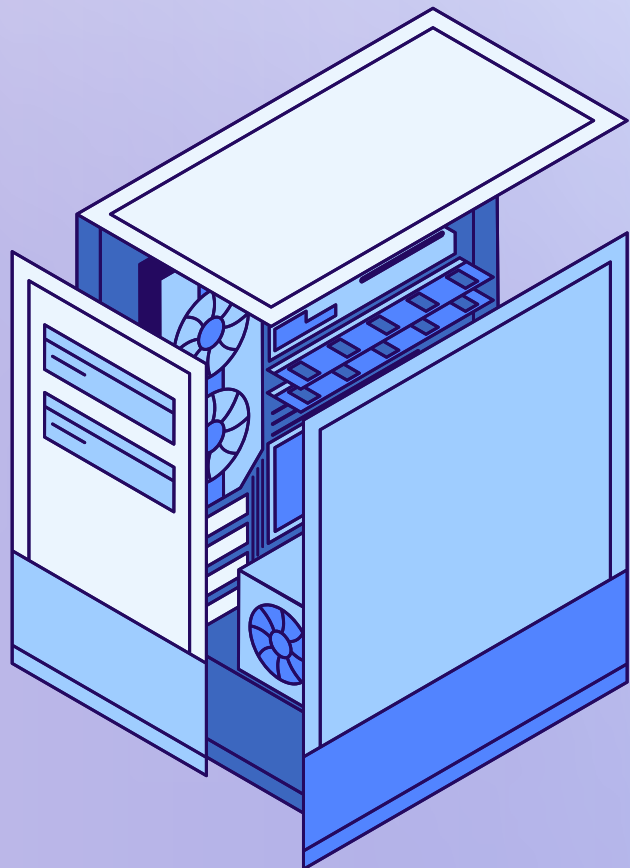
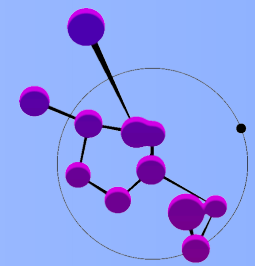
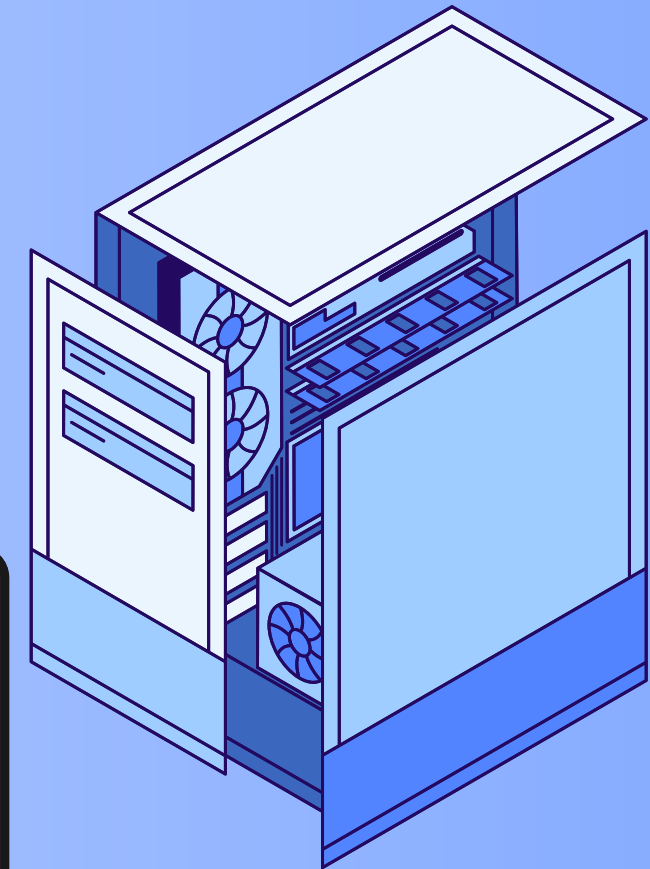
UNIT 2

Parallel Algorithm Design



PRINCIPLES OF PARALLEL ALGORITHM DESIGN: PRELIMINARIES

- Parallel Algorithm: An algorithm that executes multiple operations simultaneously using multiple processors.
- Goal: Reduce execution time and improve performance by dividing work among processors.
- Key Idea – Decomposition: Breaking a large problem into smaller independent tasks that can run in parallel.
- Task Distribution: Assigning these smaller tasks to different processors.
- Communication: Processors may need to exchange data while executing tasks.
- Synchronization: Coordinating processors to ensure correct order of execution.
- Performance Metrics: Speedup, efficiency, and scalability are used to measure performance.



DATA, DECOMPOSITION AND DEPENDENCY GRAPHS

Data

- Data refers to the input, intermediate, and output values used in an algorithm.
- In parallel computing, data is divided and processed by multiple processors simultaneously.
- Proper data organization improves performance and reduces communication overhead.

Decomposition

- Decomposition is the process of dividing a large problem into smaller sub-problems or tasks.
- These tasks can be executed in parallel on different processors.
- Types: Data decomposition and Task decomposition.

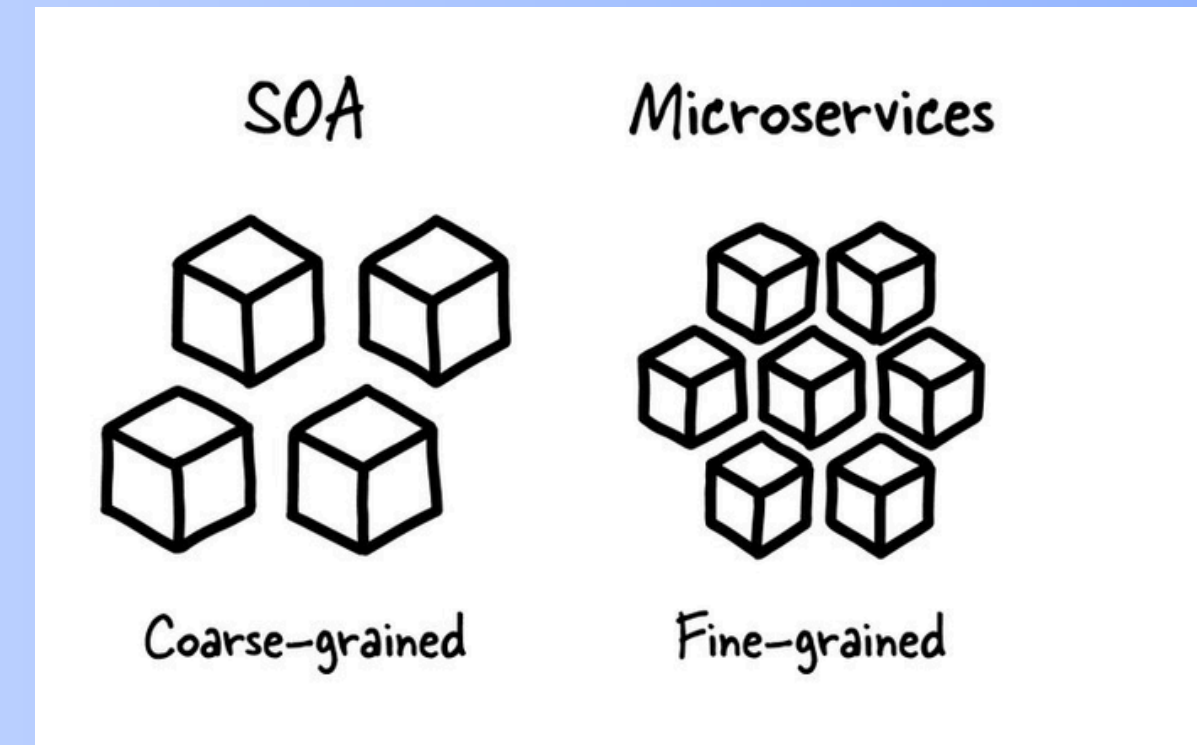
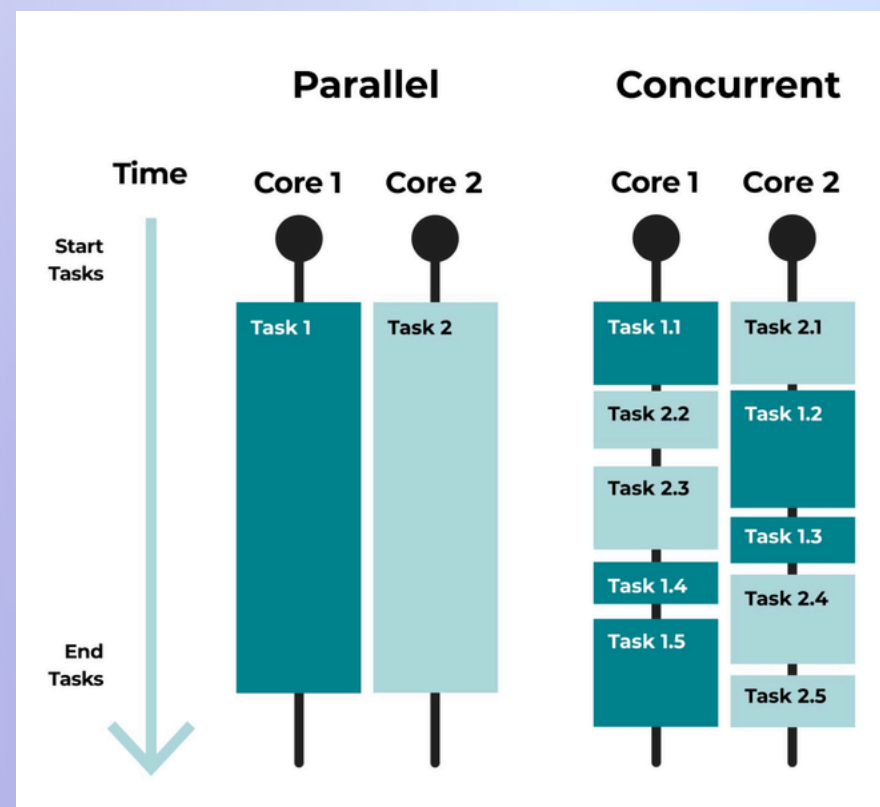
Dependency Graphs

- A dependency graph represents tasks and their dependencies.
- Nodes represent tasks, and edges represent dependencies between tasks.
- It helps determine which tasks can run in parallel and which must wait.

GRANULARITY AND CONCURRENCY

Granularity

- Granularity refers to the size of tasks in a parallel algorithm.
- It determines how much computation is done between communication events.
- Fine Granularity: Small tasks, frequent communication between processors.
- Coarse Granularity: Large tasks, less communication required.
- Choosing proper granularity improves performance and efficiency.



Concurrency

- Concurrency refers to the ability of a system to execute multiple tasks simultaneously.
- It shows how many operations can run at the same time in a program.
- Higher concurrency allows better utilization of multiple processors.
- It is an important factor for achieving parallelism.

TASK INTERACTION

- Task Interaction refers to communication and coordination between tasks during parallel execution.
- In parallel algorithms, tasks often need to exchange data with each other.
- Interaction may occur due to data dependency or shared resources.
- Proper task interaction ensures correct results and efficient execution.

Types of interaction:

- **Communication** – Tasks exchange data between processors.
- **Synchronization** – Tasks coordinate execution order.

DECOMPOSITION TECHNIQUE

- Decomposition is the process of dividing a large problem into smaller tasks that can be executed in parallel.

Types of Decomposition

Data Decomposition

- **Data is divided into smaller parts.**
- **Each processor works on a different portion of the data.**

Task (Functional) Decomposition

- **The problem is divided based on different tasks or functions.**
- **Different processors perform different tasks simultaneously.**

Recursive Decomposition

- **The problem is repeatedly divided into smaller sub-problems.**
- **This continues until the sub-problems become simple to solve.**

Speculative Decomposition

- **Multiple possible computations are executed in parallel.**
- **The correct result is selected later based on conditions.**

CHARACTERISTICS OF TASKS

Task Size

Task Independence

Task Communication

Task Synchronization

Task Creation and Termination

Load Balancing



CHARACTERISTICS OF INTER-TASK INTERACTIONS

Communication Pattern

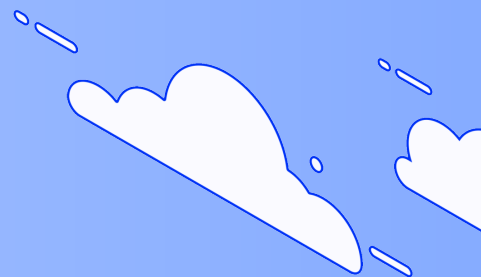
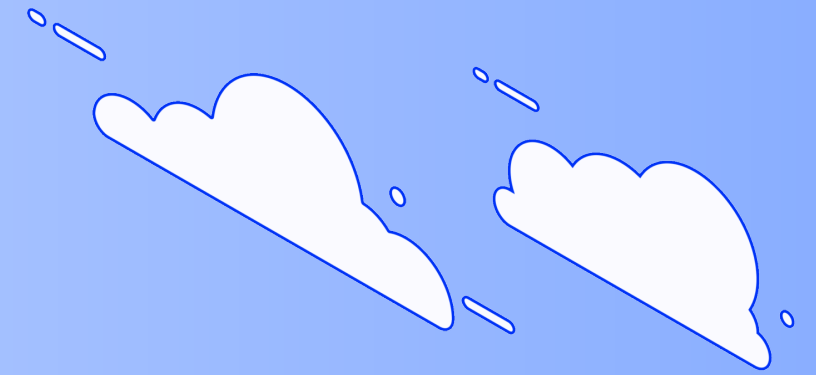
Frequency of Interaction

Volume of Data Transfer

Latency and Overhead

Synchronization Requirement

Local vs Global Interaction



MAPPING TECHNIQUES FOR LOAD BALANCING

- Mapping is the process of assigning tasks to processors in a parallel system.
- The main objective is load balancing, so that all processors perform nearly equal work.
- Proper mapping helps improve system performance and reduces idle time.

Types of Mapping Techniques:

Static Mapping

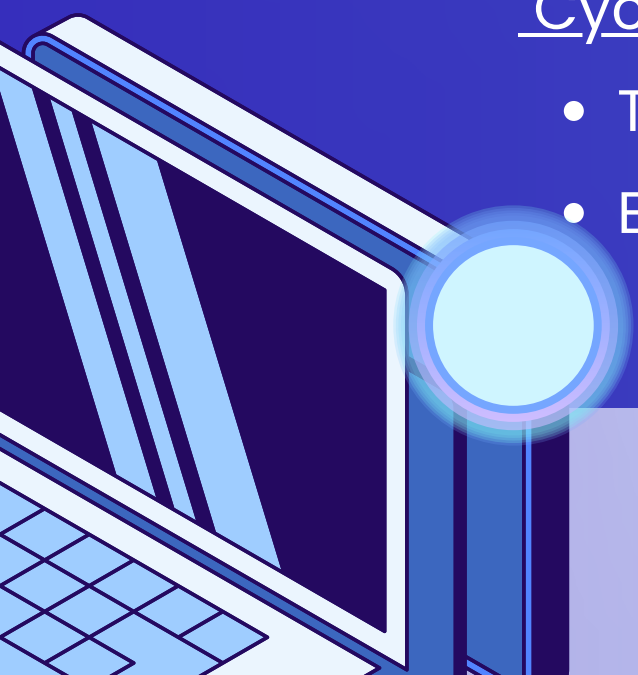
- Tasks are assigned to processors before execution begins.
- Suitable when workload is predictable.

Dynamic Mapping

- Tasks are assigned to processors during execution.
- Useful when workload is unpredictable.

Cyclic Mapping

- Tasks are distributed among processors in a round-robin manner.
- Ensures balanced distribution of tasks.



METHODS FOR CONTAINING INTERACTION OVERHEADS

- Interaction Overhead occurs due to communication and synchronization between parallel tasks.
- It can increase execution time and reduce system performance.

Some methods to reduce interaction overhead are:

Minimizing Communication

- Reduce the amount and frequency of data exchange between tasks.

Increasing Granularity

- Use larger tasks so that processors spend more time computing and less time communicating.

Efficient Task Mapping

- Place interacting tasks on nearby processors to reduce communication delay.

Overlapping Communication and Computation

- Perform computation while communication is taking place.

Reducing Synchronization Points

- Limit the number of synchronization operations to avoid delays.

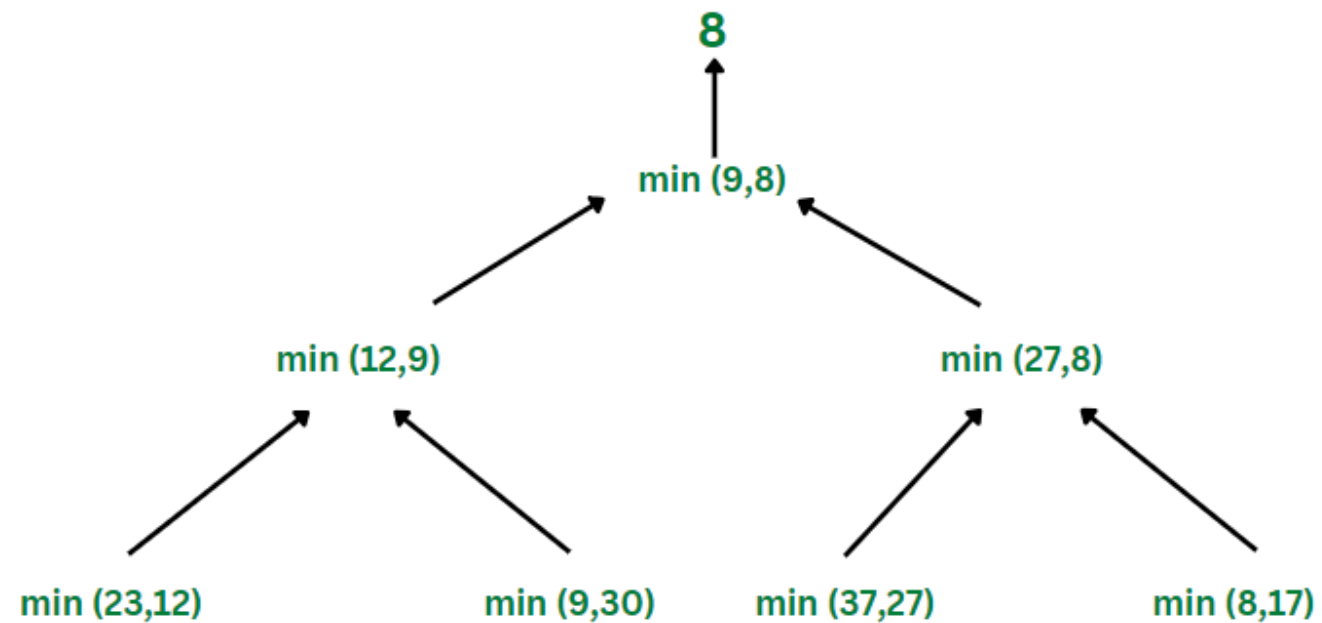
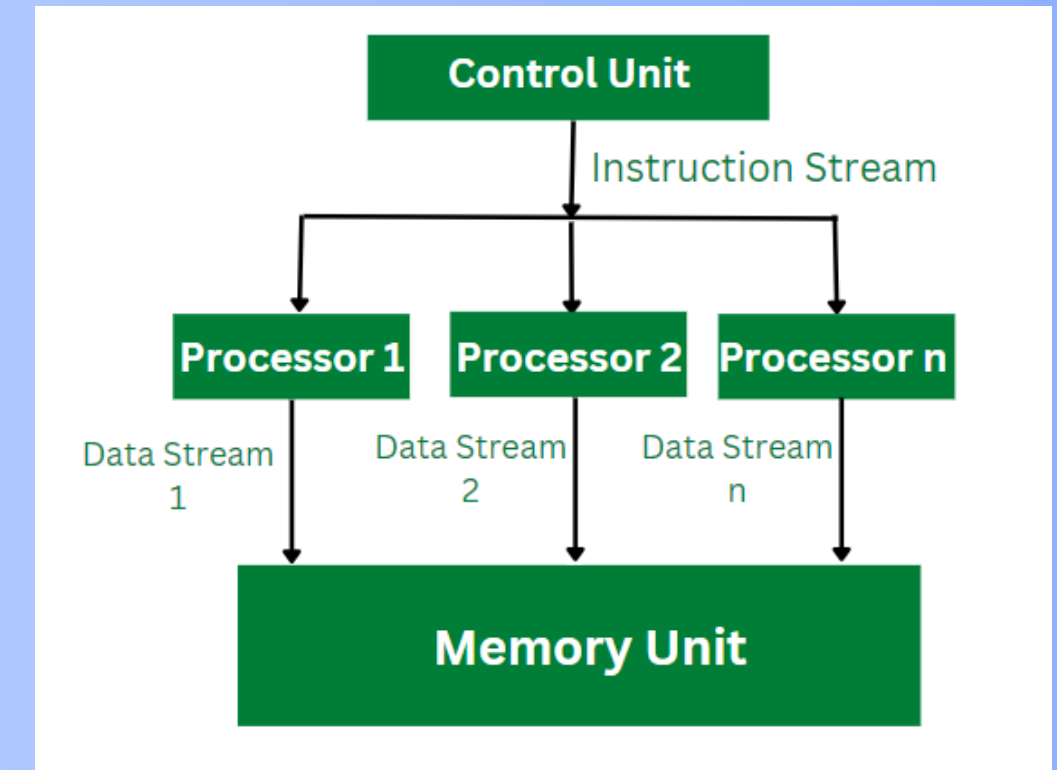


PARALLEL ALGORITHM MODELS

- Parallel Algorithm Models provide a structured way to design and analyze parallel algorithms.
- These models help in organizing tasks, communication, and computation in parallel systems.
- They simplify the development of efficient parallel programs.

DATA PARALLEL MODEL

- The same operation is performed on different parts of the data simultaneously.
- Data is divided into smaller chunks and distributed among processors.
- Each processor executes the same instruction on its portion of data.
- Commonly used in matrix operations, image processing, and scientific computing.

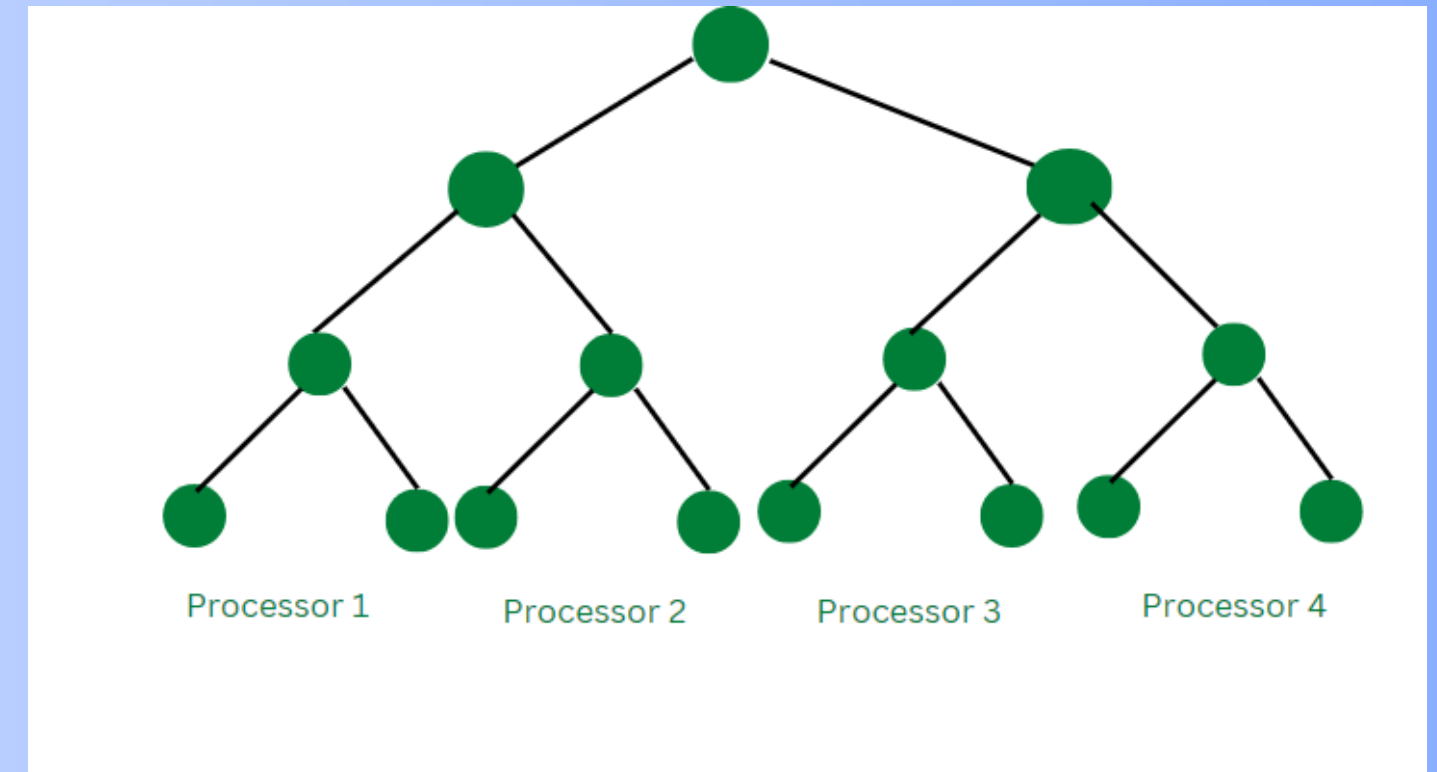


TASK PARALLEL MODEL

- Different tasks or functions are executed in parallel.
- Each processor performs a different operation on the same or different data.
- Tasks may run independently or may require communication.
- Useful when a problem contains multiple independent activities.

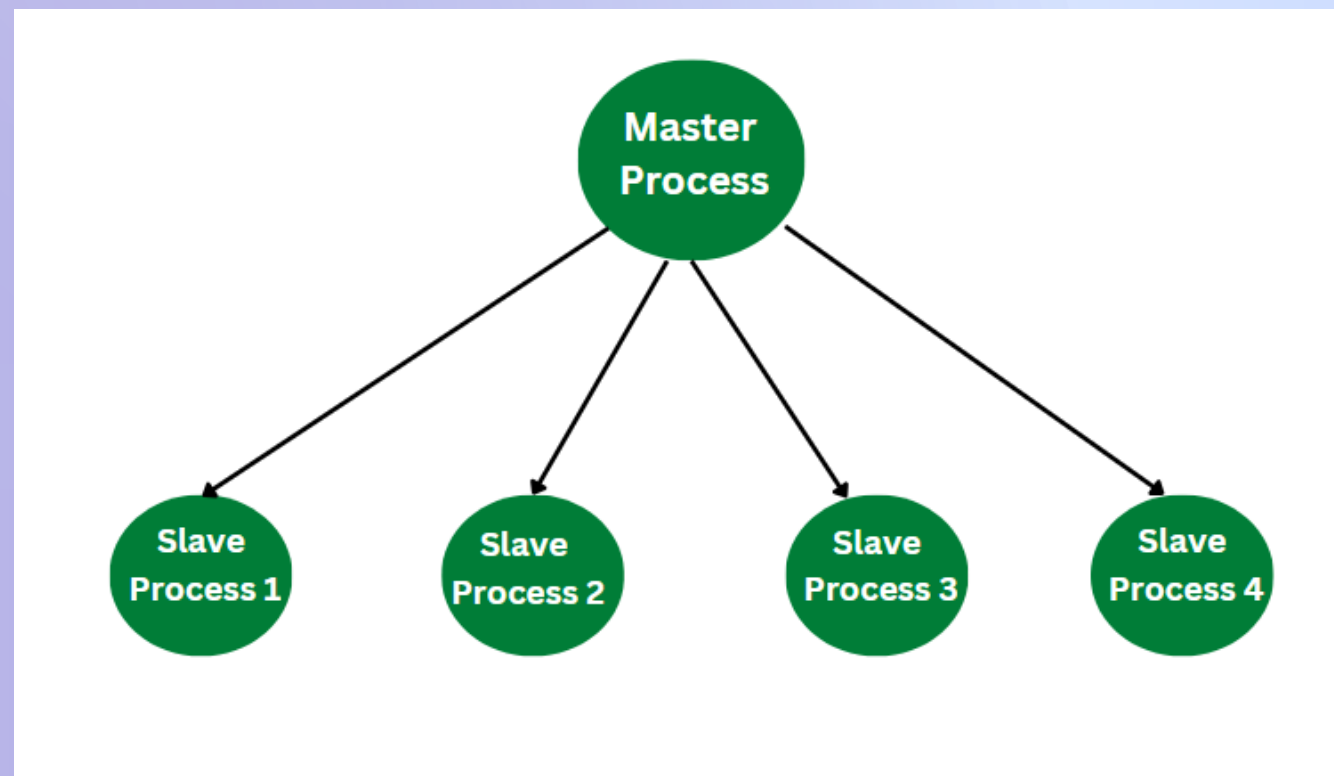
WORK POOL MODEL

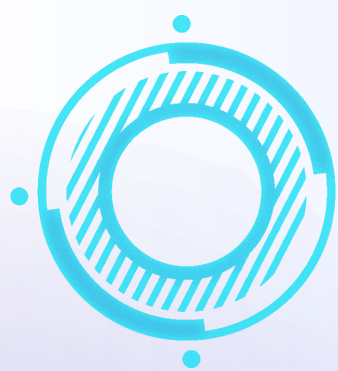
- In this model, a large problem is divided into many small tasks.
- All tasks are stored in a common work pool (task queue).
- Processors take tasks from the pool whenever they become free.
- Provides dynamic load balancing since processors keep picking tasks until the pool is empty.



MASTER-SLAVE MODEL

- One processor acts as the Master, and others act as Slaves (Workers).
- The Master processor divides the problem into tasks and assigns them to slave processors.
- Slave processors execute the tasks and send results back to the master.
- The master collects and combines the results.





SEQUENTIAL AND PARALLEL COMPUTATIONAL COMPLEXITY

Sequential Computational Complexity

- Refers to the time required to solve a problem using a single processor.
- Tasks are executed one after another (sequentially).
- Measured using time complexity such as $O(n)$, $O(n^2)$, $O(n^3)$, etc.
- Used as a baseline to compare the performance of parallel algorithms.

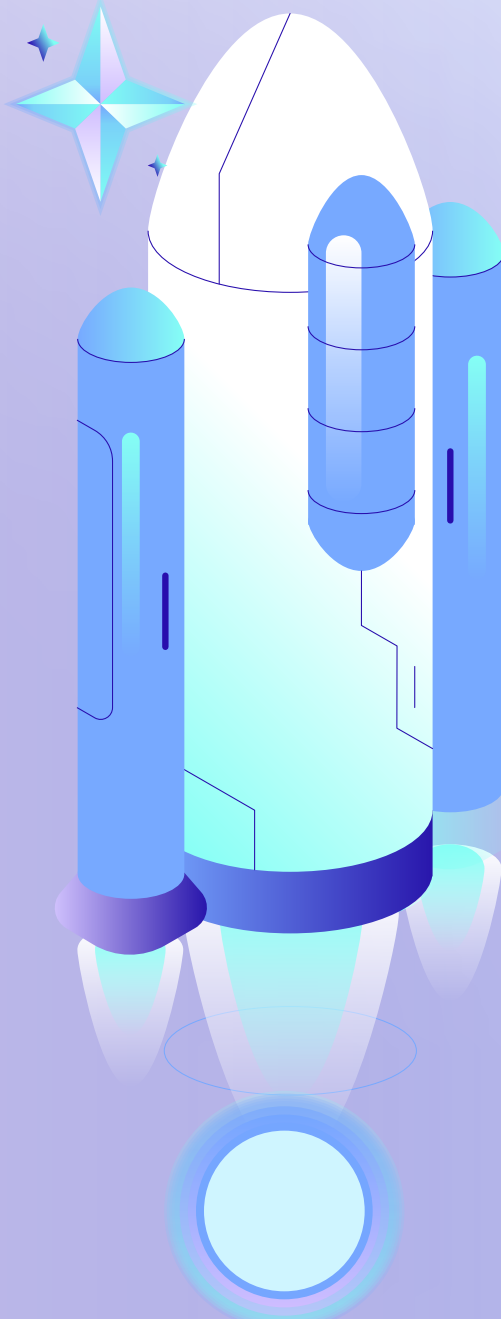
Parallel Computational Complexity

- Refers to the time required to solve a problem using multiple processors simultaneously.
- Tasks are executed in parallel, reducing overall execution time.
- Includes factors such as communication, synchronization, and processor usage.
- Measured using metrics like parallel runtime, speedup, and efficiency.



ANOMALIES IN PARALLEL ALGORITHMS

- Anomalies in parallel algorithms occur when increasing resources does not improve performance as expected.
- Sometimes adding more processors may not reduce execution time or may even increase it.
- These unexpected behaviors are called performance anomalies.

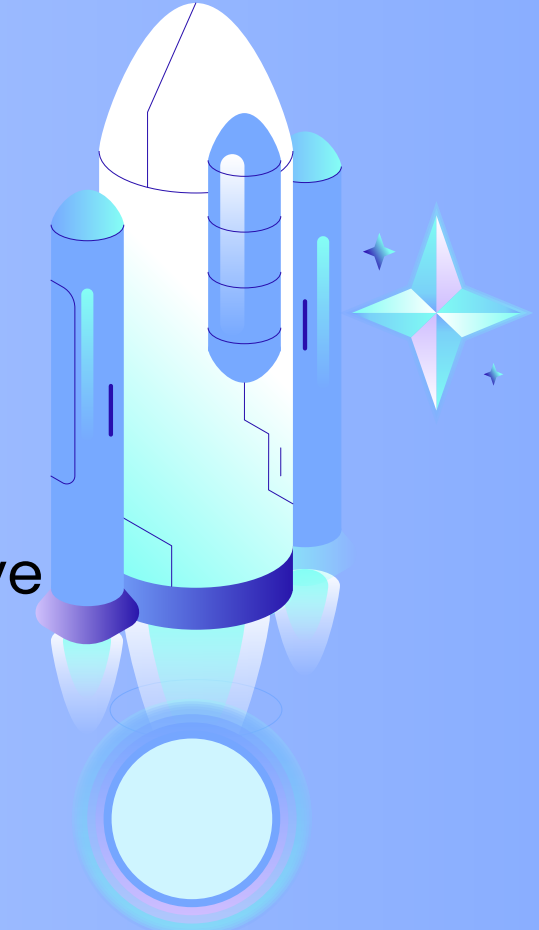


Performance
Degradation

Load
Imbalance

Communication
Overhead

Synchronization
Delay



THANK YOU!

